

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler

# Use inline backend for Jupyter
%matplotlib inline
```

```
In [2]: # Randomly oversample the minority class to match the majority class size
def random_oversample(X, y, random_state=42):
    import numpy as np
    np.random.seed(random_state)
    class_counts = y.value_counts()
    majority_class = class_counts.idxmax()
    minority_class = class_counts.idxmin()
    majority_idx = y[y == majority_class].index
    minority_idx = y[y == minority_class].index
    n_majority = len(majority_idx)
    n_minority = len(minority_idx)
    if n_majority == n_minority:
        return X.copy(), y.copy()
    additional_idx = np.random.choice(minority_idx, size=n_majority - n_minority, replace=True)
    oversampled_idx = minority_idx.union(additional_idx)
    combined_idx = majority_idx.union(oversampled_idx)
    X_over = X.loc[combined_idx].reset_index(drop=True)
    y_over = y.loc[combined_idx].reset_index(drop=True)
    return X_over, y_over

# Load and preprocess dataset: drop non-informative columns, convert target
def load_and_preprocess(url):
    import pandas as pd
    df = pd.read_csv(url)
    df.columns = df.columns.str.strip().str.replace(' ', '_')
    drop_cols = ['EmployeeNumber', 'Over18', 'StandardHours', 'EmployeeCount']
    for col in drop_cols:
        if col in df.columns:
            df.drop(columns=col, inplace=True)
    df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})
    df = pd.get_dummies(df, drop_first=True)
    X = df.drop('Attrition', axis=1)
    y = df['Attrition']
    return X, y

# Plot distribution of the target variable
def plot_attrition_distribution(y):
    import matplotlib.pyplot as plt
    import seaborn as sns
```

```

plt.figure(figsize=(5,3))
sns.countplot(x=y)
plt.title('Attrition Distribution')
plt.xticks([0,1],['No', 'Yes'])
plt.xlabel('Attrition')
plt.ylabel('Count')
plt.show()

# Plot boxplot of monthly income by attrition
def plot_monthly_income_boxplot(df):
    import matplotlib.pyplot as plt
    import seaborn as sns
    plt.figure(figsize=(5,3))
    sns.boxplot(x=df['Attrition'], y=df['MonthlyIncome'])
    plt.title('Monthly Income by Attrition')
    plt.xticks([0,1],['No', 'Yes'])
    plt.xlabel('Attrition')
    plt.ylabel('Monthly Income')
    plt.show()

# Plot confusion matrix for logistic regression
def plot_confusion_matrix(cm):
    import matplotlib.pyplot as plt
    import seaborn as sns
    plt.figure(figsize=(4,3))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No', 'Yes'])
    plt.title('Confusion Matrix (Logistic Regression)')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()

# Plot feature importances
def plot_feature_importance(importances, feature_names, top_n=10):
    import matplotlib.pyplot as plt
    import pandas as pd
    s = pd.Series(importances, index=feature_names)
    top_features = s.nlargest(top_n).sort_values()
    plt.figure(figsize=(6,3))
    top_features.plot(kind='barh')
    plt.title(f'Top {top_n} Feature Importances')
    plt.xlabel('Importance')
    plt.ylabel('Feature')
    plt.show()

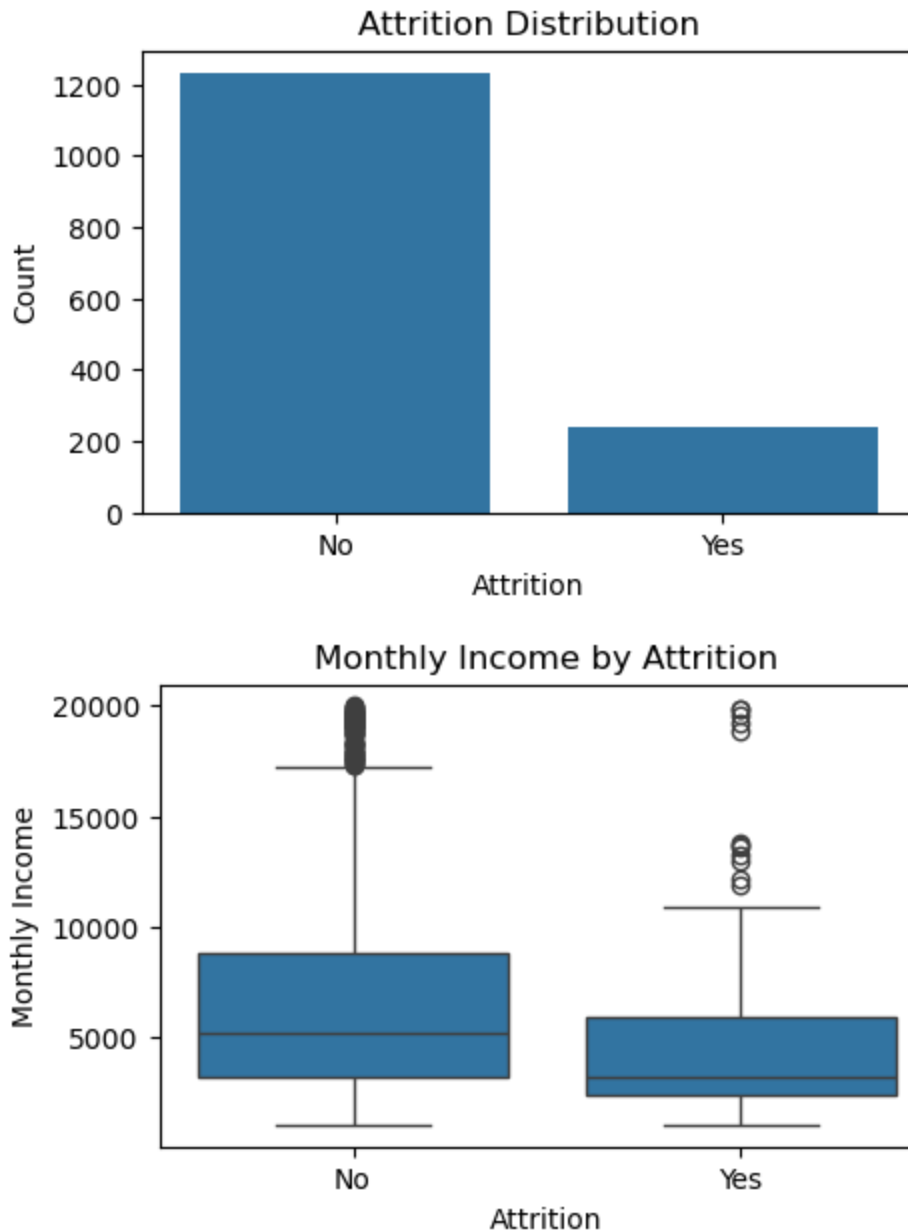
```

```

In [3]: # Load dataset and preprocess
X, y = load_and_preprocess('/Users/bryanemer/Downloads/WA_Fn-UseC_HR-Employ
# Show class distribution
plot_attrition_distribution(y)

# Load original data for boxplot
orig_df = pd.read_csv('/Users/bryanemer/Downloads/WA_Fn-UseC_HR-Employee-At
orig_df.columns = orig_df.columns.str.strip().str.replace(' ', '_')
orig_df['Attrition'] = orig_df['Attrition'].map({'Yes': 1, 'No': 0})
plot_monthly_income_boxplot(orig_df)

```



```
In [4]: # Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, str

# Oversample minority class in training data
X_train_over, y_train_over = random_oversample(X_train, y_train)

# Standardize features
scaler = StandardScaler().fit(X_train_over)
X_train_scaled = scaler.transform(X_train_over)
X_test_scaled = scaler.transform(X_test)

# Define models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, n_jobs=-1),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=4
}
```

```

results = {}
conf_matrix = None

for name, model in models.items():
    model.fit(X_train_scaled, y_train_over)
    preds = model.predict(X_test_scaled)
    proba = model.predict_proba(X_test_scaled)[:,1] if hasattr(model, 'predict_proba') else None
    results[name] = {
        'Accuracy': accuracy_score(y_test, preds),
        'Precision': precision_score(y_test, preds),
        'Recall': recall_score(y_test, preds),
        'F1-score': f1_score(y_test, preds),
        'ROC-AUC': roc_auc_score(y_test, proba) if proba is not None else None
    }
    if name == 'Logistic Regression':
        conf_matrix = confusion_matrix(y_test, preds)

# Display metrics
import pandas as pd
metrics_df = pd.DataFrame(results).T
metrics_df

```

Out [4]:

	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.795918	0.410959	0.638298	0.500000	0.809200
Decision Tree	0.741497	0.177778	0.170213	0.173913	0.510208
Random Forest	0.833333	0.437500	0.148936	0.222222	0.781549

In [5]: *# Plot confusion matrix for logistic regression*
plot_confusion_matrix(conf_matrix)

```

# Plot feature importances from random forest
rf_model = models['Random Forest']
feature_importances_ = rf_model.feature_importances_
plot_feature_importance(feature_importances_, X.columns.tolist(), top_n=10)

```

